Project Management in Action

From The Executive Suite

THE REWORK CYCLE: Why Projects Are Mismanaged

Kenneth G. Cooper *is director of the Management Simulation Group and senior vice president of Pugh-Roberts Associates, a division of PA Consulting Group. His management consulting career spans 20 years, specializing in the development and application of computer simulation models to a variety of strategic business issues. His clients include AT&T, Aetna, Arizona Public Service, Hughes Aircraft, IBM, Litton, MasterCard, McDonnell-Douglas, Northrop, Rockwell, and several law firms. Mr. Cooper has directed over a 100 consulting engagements, among them analyses of 60 major commercial and defense development projects. He is an original author of the program management model introduced in this article. His group's offices are in Cambridge, Massachusetts, and Oxford, England. Mr. Cooper received his bachelor's and master's degrees from M.I.T. and Boston University, respectively.*

The project was slated to design and build an exciting first-of-a-kind product. But during the development effort, unexpected problems emerged—changes in the design specifications, shortages of qualified people, material supply delays. Costs escalated, and work fell far behind schedule. The project's future was threatened, and the work was interrupted. Eventually, project objectives were scaled back and the work was completed—years late and at more than double the original budget.

If this tale of development project woes sounds all too familiar, perhaps it is from personal experience, or because others like it are being lamented so frequently in company boardrooms, and reported in business publications with disturbing regularity. It might be a construction firm's experience in building a power plant, or a software company's troubles in bringing new systems to market, or the most recently-reported defense program problems, or the automobile industry's struggle to cut product development time.

But it isn't. The product sponsor in this tale was George Washington; Paul Revere supplied the required copper and brass fittings. The special timber that delayed the work's progress eventually provided the project's product with its nickname, "Old Ironsides." The U.S.S. Constitution inaugurated the U.S. Navy and was, in 1794, a new nation's introduction to the challenge of large development project management. Today the challenge remains largely unanswered.

The simple fact is that the traditional art/science of complex development project management is a failure. The consequences of this failure are enormous. Contractors and other businesses lose vast sums of money on the development efforts themselves. The costs of the nation's defense soar. Legal disputes over contracted project cost responsibility drag on for years. Other projects are denied scarce resources. Products are late to market, at higher prices, and with fewer features than planned and promised. Market shares, jobs, and profits are lost. Our competitiveness as companies and as a nation suffers.

> *... CPM ... [and] earned value are inadequate models for managing complex development projects.*

Why have we made so little progress in learning how to manage large projects? Why, with all the project management systems and tools—computerized and otherwise—do we keep getting "surprised"? Is every large project just "too unique" to extract and transfer lessons from one to another? Or is there enough commonality among such projects that we may learn, improve our tools, and perform to higher standards?

THE BLANK TAPE-MEASURE

Imagine that you have contracted for your house to be built, and that the standard tape measure used by all the contractors has long segments—half the tape, in fact—that are uncalibrated. What sort of product would you expect from their efforts?

We are, in effect, equipping project managers with half-blank tape measures as the standard tools for planning, monitoring, and managing large projects. With my colleagues, I have analyzed in depth over sixty major development projects and programs in aerospace, large construction, software systems, shipbuilding, defense electronics, and telecommunications. When we began doing so over ten years ago, we were asked by our contractor client to build a computer-based model capable of accurately simulating the performance of a large project from the start of design through the completion of construction. Large portions of the project effort simply could not be described or explained by applying the standard available tools. How could this be?

The Critical Path Method (CPM) has long dominated among techniques for project planning. This method provides a framework in which the duration of, and linkages between, individual tasks can be planned. From this, the sequence of tasks may be identified which, if one element on the path were to be delayed, would translate to a delay in the entire project. It is an accepted, often required, technique for planning projects and for testing schedule impacts. It is the basis for virtually every piece of popular project management software offered. Properly constructed and updated, it is an extremely useful planning tool. And yet CPM is an inadequate model for managing complex development projects.

The typical means for monitoring project progress and ongoing cost and schedule performance are variations of *earned value systems*. These provide for setting work and budget standards for individual tasks. Progress on the tasks, and cost and schedule variations, are assessed by comparing actual effort and cost with the task budgets. The earned value system for project monitoring is, like CPM, an accepted and often contractually required project management technique. Truthfully and faithfully employed, it is a highly disciplined monitoring method. And, like CPM, earned value is an inadequate model for managing complex development projects.

For decades there has been a stagnation in the prevailing theory and practice of large project management. "Advances" have been largely confined to the computerization of fundamentally inadequate methods. So what's missing?

THE MISSING MEASURE

What is missing, as any experienced project or program manager knows, is *rework*. For all their utility, conventional methods treat a project as being composed of a set of individual, discrete tasks. Each task is portrayed as having a definable beginning and end, with the work content either "to be done" or "in process" or "done." No account is taken of the *quality* of the work done, the release of incomplete or imperfect task products, or the amount of rework which will be required. This is particularly inappropriate for development projects, in which there is a naturally iterative process of design/engineering.

Indeed, our analyses have shown that **rework can account for the *majority* of work content (and cost) on complex development projects**! While this varies significantly among projects and project types, it is hardly ever a matter of a single re-visiting of a particular task. Instead, several iterations are

What we need, then, is a different **view** *of development projects, one which recognizes the rework cycle, plans for it, monitors it, and helps managers reduce its magnitude and duration.*

typical, often far removed in time from the scheduled and actual conduct of the first round of work on the task. This is readily seen in, for example, the release of initial engineering drawings, "A" revisions, "B" revisions, "C" revisions (for those companies or projects which actually monitor such information). Companies experienced in complex projects know to expect this, and have developed rules of thumb to count on 2 (or 3 or 4 ...) revisions per engineering product. Even so, this expectation rarely is incorporated explicitly in work planning and management systems—because the techniques don't allow it. Worse are the cases where this rework cycle is **not** explicitly anticipated or monitored. Here they are not only working with half a tape measure—they're reading it with their eyes closed! These are not unintelligent people nor project-naive companies; on the contrary, they include the most technically sophisticated individuals conducting and managing complex developments in firms whose very existence depends on successful project performance.

What we need, then, is a different **view** of development projects, one which recognizes the rework cycle, plans for it, monitors it, and helps managers reduce its magnitude and duration. We need a method that reflects a more *strategic* view of projects, one which accounts for the *quality* of work done and the *causes* of productivity and rework variations. We need to be able to see more clearly than is allowed by "traditional" methods how changing external conditions and our own management actions alter staff productivity and the rework cycle—and how the consequences spread through an entire project. We need a new framework applicable across a range of projects, reflecting that which is common and that which is unique among projects. Only thus may we more consistently and rigorously extract, learn and apply lessons that will yield sustained improvement in project management and performance.

Such a new framework has emerged from the application of System Dynamics simulation methods to a wide range of development projects. In a manner quite dissimilar to CPM/PERT models, it treats a project not merely as a sum or a sequence of discrete tasks, but as *flows* of work in which there are multiple rework cycles. Because of the significant rework content of development projects, this framework is able to reconcile with one another a project's person-hours spent, tasks/items performed, elapsed time, and much more. Not only can the Rework Cycle model structure accurately simulate the actual recorded history of projects, but it can provide powerful forecasting and "What if" managerial capabilities!

First built for a ship design project at Litton*, the Rework Cycle model has since been applied accurately and successfully to over sixty different projects—software system developments at AT&T, defense electronics systems at Hughes Aircraft, the Cross-Channel Tunnel (constructed by the Transmanche-Link consortium of contractors), aircraft design and production at Northrop, electric utilities' power plant engineering and construction, and dozens of other programs and projects.

At the core of the structure is a different but straightforward view of project work accomplishment—one which recognizes the rework cycle. That simple addition to the traditional view has, however, provided an extremely powerful diagnostic and management capability for project managers in the many organizations with which we've tested its application.

The structure of the Rework Cycle model and associated lessons are described in the accompanying Issue Focus article later in this issue. Empirical results and guidelines usable by project managers will be presented in the March 1993 issue of the *Project Management Journal.*

*Naval Ship Production: A Claim Settled and a Framework Built. Cooper, K.G. *Interfaces*, a publication of The Institute of Management Sciences, December 1980. PMI

This article is reprinted from the February 1993 *PM Network* magazine with permission of the Project Management Institute, Four Campus Boulevard, Newtown Square, PA 19073, USA, a worldwide organization for advancing the state of the art of project management. Phone 610/356-4600

Concerns of Project Managers

Issue Focus

THE REWORK CYCLE: How It Really Works ... And Reworks ...

Editor's Note: It is the editor's opinion that the concepts presented in "The Rework Cycle" are of such significance to understanding the nature of development projects that it is being presented in three articles, two in this issue of the *PMNETwork* and one in the March issue of the *Project Management Journal*. In "From the Executive Suite" the concept is introduced. In this article, the conceptual model is presented. In the *PMJ* article, a more detailed discussion of the rework phenomenon provides insights which lead to more realistic planning of developmenal projects. In particular, and substantially enhances the normative model of projects as presented to date in the PMBOK (Project Management Body of Knowledge) and other sources.

Kenneth G. Cooper, Pugh-Roberts Associates/PA Consulting Group, Cambridge, Massachusetts

PMs

In order to achieve a more strategic view of projects, more effective management, and consistently better performance, the *rework cycle* must be a part of our understanding of development projects, and recognized in the systems used to help manage those projects. Like bricks without mortar, methods and systems which fail to do so cannot stand soundly, and project-based businesses will continue to fail consistently in meeting their competitive goals in cost, schedule, and quality performance.

Based on the experience of analyzing dozens of complex development projects, we have developed a structure for significantly enhancing the traditional view of a project. Repeated applications of this more realistic "model" have proven it to be logically correct and, when codified as a working simulation model, numerically accurate. Its uses have brought benefits (as in dollars) that are large (as in billions) to the businesses that have adopted it. The important core of the model structure **is** "the rework cycle."

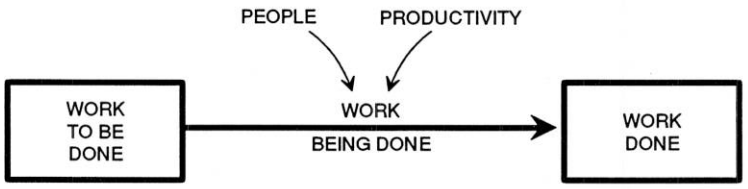
THE TRADITIONAL VIEW OF A PROGRAM

First, re-casting the traditional view of a project's (or project stage's) **tasks to be done**, **tasks in process**, and **tasks done** as a more continuous stream of work ...

1. The Traditional View, as a Flow of Work

The pool of tasks in **work to be done** is depleted over the course of time, such that at the end of the project, nothing is left there, and all the tasks fill the pool of **work done**.

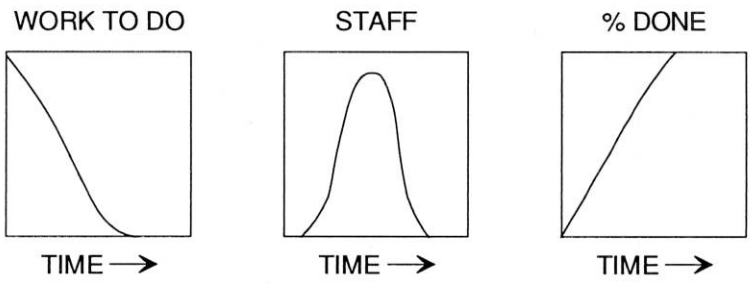
Taking this diagrammed "model" a small step further by recognizing that it is **people** working at some (varying) level of **productivity** that causes the work to get done ...



2. Explicit Productivity Affects the Pace of Work Flow

Changing the number of people along the way, or somehow influencing their productivity, alters the pace of work getting done.

Let us look at the shape of some key measures of project performance that would occur under this traditional view. Plotted below are graphs of "Work To Be Done," project (stage) staff, and percentage complete, as computed by a simulation model using the diagrammed structure ...



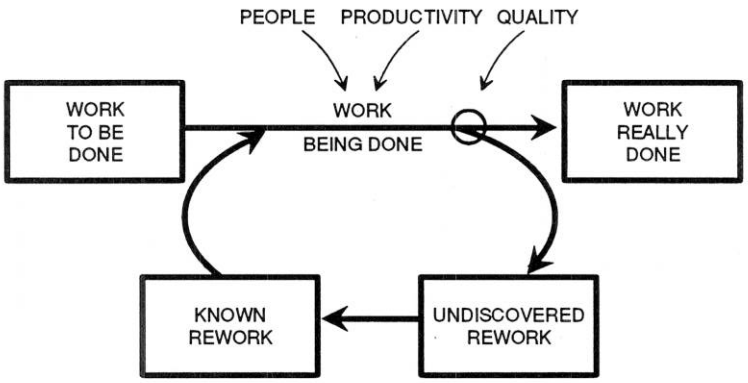
3. Simulated Performance of a Project, Using the **Traditional View**

These may show the way we *plan* the effort to go; this may be the way we *hope* things will go. But let's be honest—how many of us have seen even a remotely ambitious development project actually perform this way?

A BETTER VIEW

A better view recognizes the existence of rework cycles. Below, what is termed the **quality** of work executed should be thought of as a "valve" controlling the portion of the work flow being done that will or will not require **rework** ...

View the diagrammed structure as physical *pools* in which work resides and *pipes* through which work flows. It's easy to see that a "quality" measure that could vary (over time) in the range of 0 to 1.0 diverts more or less of the work being done into the rework cycle. So long as this measure of quality is less than 1.0, some work being done—even rework itself—will continue to move into and through the rework cycle.



4. Explicit Rework Demonstrates the Cycle

The distinction drawn between **productivity** and **quality** is important. Staff may exhibit high "productivity," but be putting out work of low "quality" that requires later re-working. In this condition, the net throughput to the pool of **work really done** is low[1].

The pool of rework requires staff to expend effort to execute it—to alter/correct/complete the work items needing revision. With this addition to the model, different project performance would occur. As shown in the plots below, somewhat more familiar and realistic patterns are simulated when rework is generated and executed ...

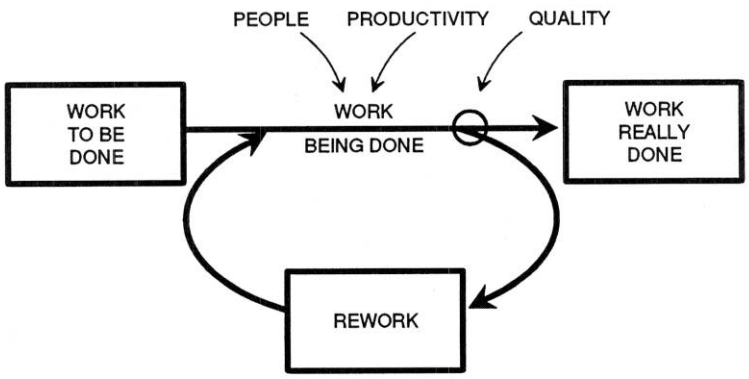
Recognizing the allocation of additional staff effort to execute rework, and the resulting slowdown in the pace of final completion, provides an accurate description of work on a project ... almost.

5. Simulated Performance of a Project, With **Explicit Rework**

THE REALITY

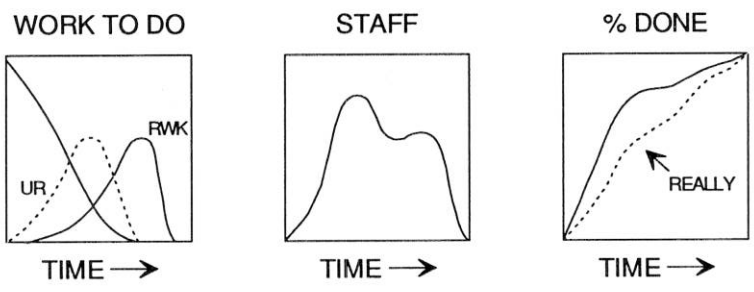
In reality there is a critical "way station" in which elements of rework linger until identified as *needing* rework. We have termed this way station **undiscovered rework** ...

Undiscovered rework consists of those tasks or work products that contain as-yet-undetected errors of commission or omission, and are therefore perceived, *and reported by all traditional systems*, as being **done**.



6. The Rework Cycle, Complete With Undiscovered Rework

The completed model of the rework cycle yields simulation-generated behavior that is characteristic of all development projects …



7. Simulated Performance of a Project, With the Full Rework Cycle

The precise quantities and timing obviously differ among projects, but the behavior is common: as the initial round of work nears conclusion, previously undiscovered errors become apparent, requiring more staff for a longer time; the perceived and reported progress significantly slows as the magnitude of recognized rework grows, and an extended completion effort ensues as the last elements of undiscovered rework emerge.

Most rework is discovered by "downstream" efforts or testing, but months (or even **years**) may pass before this discovery occurs! During this time dependent work will have incorporated these errors, or technical derivations thereof, and enter its own rework cycle. The more tightly-scheduled and parallel the project tasks, the more of a "multiplier effect" on subsequent rework cycles.

This final element of the rework cycle, undiscovered rework, plays a pivotal role in the propagation of problems through a project. Lurking undetected—as a software "bug," or design miscalculation, or wrongly-placed bulkhead—it causes productivity loss, delays, and *more* rework on dependent tasks. Undiscovered rework is *the single most important* source of project cost and schedule crises. It is the great killer of projects (and of new products and of careers), and no traditional systems even acknowledge its existence. Even more important, most projects seem to be planned and managed as though it does not exist.

The specific technical content of undiscovered rework is by definition unknown at any point in the program. But it is imperative to program management success that it be:

(a) ***Acknowledged*** (and plans and schedules set accordingly so as to reduce the disruption of the "surprise");
(b) Actively ***sought out*** (while earlier discovery may feel unpleasant at the time, it is important not to "kill the messenger"—rather, to *encourage* early technical problem identification. Here, it *is* what you don't know that hurts you most.);
(c) ***Prevented*** as much as possible, i.e., improving "quality" as defined here (easier said than done, since managers cannot mandate by edict the achievement of higher quality).

Since the approach advocated here is **not** common practice, and most companies and managers do not have the benefit of historical data for reference, what should be expected of a project's work "quality," rework detection times, the magnitude of undiscovered rework? What is "good"? Is more rework found early helpful, or the harbinger of a disaster to come?

In the upcoming (March) issue of the *Project Management Journal*, the final installment of this article series will draw upon dozens of large development project analyses to present our findings on …

- How to measure your development projects' "quality" (it's easy if you measure the right stuff)
- What values of quality are "good" among different project types (you'll see how it varies—one type's ceiling is another type's floor)
- The project cost and schedule effects of achieving different "quality" levels (they're big)
- How to estimate your "rework cycle time" (again, "easy")
- The value achieved by accelerating the detection of undiscovered rework (it's non-linear)
- How to attain much more accurate assessments of real project progress (forewarned is forearmed)
- Some implications for project management process improvement (and cost and quality and schedule improvement)

- And some specific lessons from a couple of project "war stories."

CONCLUSIONS

Nothing short of a fundamental change in the way managers view development projects is required. To avoid the persistent cost and schedule performance problems so closely associated with complex developments, we must take a more strategic and realistic view of project work content and processes. We must recognize that while the products and technical steps may indeed be unique, there **are** common structures and processes, and common problem causes. From these, it is possible to extract lessons and to implement changes that achieve radical improvements in project performance and business success.

A development project is not merely a sum or a sequence of discrete tasks, but contains **flows** of work in which there are multiple *rework cycles*. Executing these rework cycles often requires the bulk of the project effort, rendering conventional systems that ignore rework deficient and misleading.

Of all the aspects of the rework cycle, the most malignant is *undiscovered rework*. Not only does it have its own cost in time and resources, but it is responsible for spreading problems through dependent later (usually more expensive) project stages. We must acknowledge, measure, actively seek out, and where possible **prevent** undiscovered rework, and the rework cycle, through management initiatives. Successful initiatives will combine changes in the "culture" of the organization, the management support systems, the work practices, the monitoring and scheduling logic, and the way we relate to superiors and customers. We cannot control by mandate the "levers" of quality and rework detection. Instead we need to *influence* them indirectly through that which we **can** control, or more directly influence—interim schedule targets, staffing, monitoring systems, testing practices.

The more proactive managers will achieve the greater impact and success. With more accurate information and expectations, they will have the most advance warning, the most time in which to pose and examine "What if" scenarios, the most considered and carefully-crafted actions, the most highly-leveraged improvements achieved.

Our analyses show the range of possible values for work "quality" and rework discovery times within each project type to be sufficiently broad that **dramatic** improvement is achievable. For globally-competitive businesses in technology-intensive industries, such improvement in project management and performance is vital. The alternatives are unacceptable—unforeseen costs, unexpected delays, unfulfilled promises—all products of the rework cycle.

NOTE

1. This distinction between productivity, quality, and rework has the added benefit of making all of these factors measurable and monitorable. Total throughput of work items in a project stage (lines of code, tons of steel, drawings, numbers of units, tests conducted, or ...) can be measured over time much as in traditional systems, and compared to the number of hours spent in the same time frames, so as to monitor a legitimate measure of productivity. Numbers of revisions and rounds of revisions, can be monitored over time so as to derive a tangible measure of quality, as described in the companion article in the March 1993 issue of the *Project Management Journal*.

Pugh-Roberts and Associates
PA Consulting Group
41 William Linskey Way
Cambridge, MA 02142

Phone (USA): (617) 864-8880
Fax (USA): (617) 864-8884
Phone (UK): (865) 728-898
Phone: (UK): (865) 728-882

Author's Note: *The occasion of reprinting affords me an opportunity that space limitations in the original publication denied: The explicit acknowledgement of individual clients and colleagues. Without their hard and often brilliant work and enthusiastic support, the body of work on which I have drawn these articles would not have been as important, nor as much outright fun. In roughly chronological order, my sincere thanks go to Henry Weil, Richard Goldbach, Ed Roberts, Jack Pugh, John Kuelbs, Jim Lyneis, Art Cohen, Rich Park, Bill Dalton, Craig Stephens, Frank Burger, Dick Marler, Rayford Etherton, Mike Miller, Tom Kelley, Jane Hemingway, Jennifer Broadhurst, Tom Mullen, Todd Sjoblom, Kim Reichelt, Sharon Els, Carl Bespolka, Mike Moberly, Bob Metzger, Thierry Chevally, Prima Soemijantoro, and Fran Webster — a mix of colleagues and clients and friends all; their contributions deserve more than a simple list. Lifelong thanks to my parents for helping me to believe, and to Melissa Day Cooper for the inspiration.*

This article is reprinted from the February 1993 *PM Network* magazine with permission of the Project Management Institute, Four Campus Boulevard, Newtown Square, PA 19073, USA, a worldwide organization for advancing the state of the art of project management. Phone 610/356-4600

THE REWORK CYCLE: Benchmarks for the Project Manager

Kenneth G. Cooper, Pugh-Roberts Associates/PA Consulting Group, Cambridge, Massachusetts

Editor's Note: *The following article is the third in a three-part series, the first two of which are in the February 1993 issue of PMNETwork. There the concept and workings of the rework cycle are introduced. Based on dozens of applications to major development projects, the structure portrays flows of project work in which there are multiple cycles of rework. Rework typically represents the bulk of development project expenditures and time. The absence of its treatment in conventional methods and systems is a critical omission which consistently leads to project overruns. This article translates experience using the rework cycle "model" into practical suggestions and guidelines for use by managers of development projects.*

When gauged by initial expectations for cost, schedule, and planned product, the prevailing modes of performance on complex development projects are (1) surprise and (2) failure. Conventional methods and systems have contributed to the consistently poor track record of complex project performance. Critical-path-based methods lack any consideration of the need for reworking incomplete tasks. Earned-value systems, even those endorsed or required by the government, have been likened by our contractor clients to driving a car by watching the rearview mirror. We have developed and applied extensively a structure designed to portray and anticipate the *rework cycle* in development projects.

In the diagram below, the boxes represent pools of work (drawings ... lines of code ... feet of cable ...).

At the start of a project or project stage, all work resides in the pool of **work to be done.** As the project begins and progresses, changing levels of staff (**people**) working at varying **productivity** determine the pace of **work being done**. But unlike all other program/project analysis tools and systems, the rework cycle portrays the real-world phenomenon that work is "executed" at varying, but usually less than perfect, **quality.** Potentially ranging from 0 to 1 , the value of quality (as well as that of productivity) depends on many variable conditions in the project and company. The fractional value of quality determines the portion of the work being done that will enter the pool of **work really done**, which will never again need re-doing. The rest will subsequently need some rework, but for a (sometimes substantial) period of time the rework remains in a pool of what we term **undiscovered rework**—work that contains as-yet-undetected errors, and is therefore perceived as being done. Errors are detected by "downstream" efforts or testing; this **rework discovery** may occur months or even years later, during which time dependent work has incorporated these errors, or technical derivations thereof. Once discovered, the **known rework** demands the application of resources, beyond those needed for executing the original work. Executed rework enters the flow of work being done, subject to similar productivity and quality variations. Even some of the re-worked items may then flow through the rework cycle one or more subsequent times.

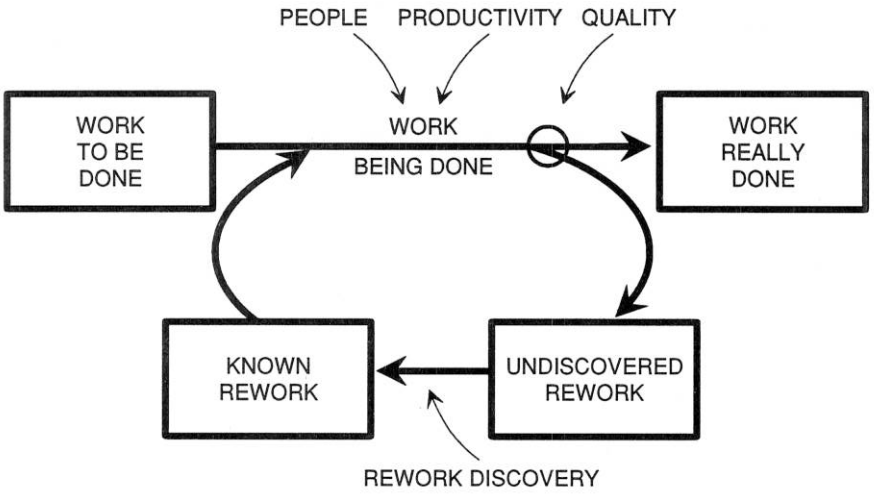


Figure 1. The Structure of the Rework Cycle.

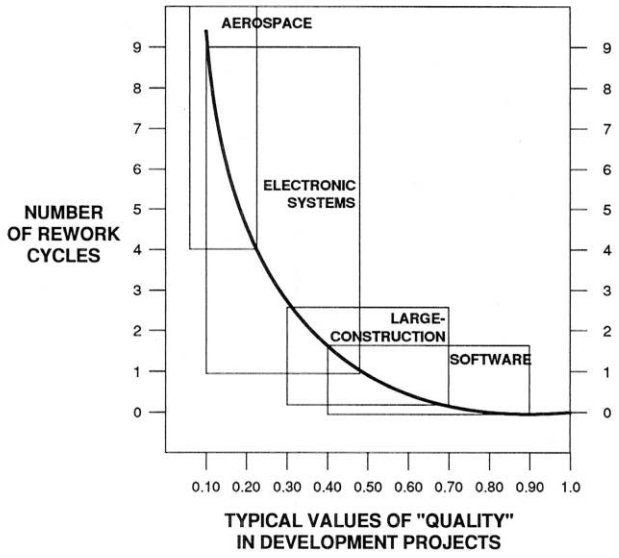
For several years my colleagues and I have been using simulation models[1] based on the rework cycle, and adapting them to accurately re-create, forecast, and diagnose the performance of each of a variety of projects. Across the projects, and within certain groups of projects, have emerged patterns of behavior which can be useful as managerial rules of thumb. While each project is indeed different, the empirical observations and measurements reported here are valid benchmarks for all those who undertake to manage the rework cycle in their own projects.

THE QUALITY RANGE

First, to underscore the magnitude of the issue (and the appropriate degree of managerial concern), consider the range of values exhibited for "quality"—the fraction of work being executed that will not require subsequent rework. As indicated in the display box, the effective values range from as high as 0.90 all the way down to below 0.10. The chart shows the range of "quality" values for each type of project[2], and the resulting number of rework iterations.

Of the projects examined, commercial software development projects exhibited the

PMBOK References:
Z.7.d
B.6.b
C.3.d.II
E.3.a



QUANTIFYING QUALITY

This chart shows the empirically-derived value of "quality"—the fraction of work being executed that will not require subsequent rework—for dozens of real development projects in different arenas. Along with the range of quality for each industry segment are shown the rework cycles implied by that range. As lower quality causes more cycles of rework, more effort (cost) must be expended to complete the work—the equivalent of doing it once, and then doing it all over again and again ... A quality of 0.20 will require five cycles of work and cost (four full rework cycles) to "get it right."

Development projects here are those which are dominated by design/engineering activity to develop a new product or system, and do not include the building of units to a stable design.

lowest amount of rework (highest "quality"), ranging from little rework in some stages to "only" one and one-half full rework cycles in others. (Many such projects are actually adaptations of prior developments.) At the other extreme are aerospace development programs—all of these are advanced military developments, usually with substantial research efforts—which typically have at least four (and usually more) rework cycles in the design effort. Between these two extremes are electronic systems development projects (typically designing and integrating systems of new hardware and new software) which exhibit one to nine full rework cycles, and the design of large-construction projects, with a range of about one-half to two and one-half rework cycles[3].

Clearly the larger the technological leap being attempted by the overall project, the lower this measure of "quality" will be (the more rounds of rework). But even within the range of each project type, there is considerable variability in work quality and, therefore, in the cost and time required in a development project. Consider the project performance improvement achievable with even moderate improvements in quality! In an era when 30-40 percent performance improvements are set forth as ambitious targets for organizations undergoing "change programs" and "process reengineering," the highly-leveraged impact of this kind of "quality improvement" must not be overlooked.

THE REWORK DISCOVERY TIME

Still, no matter what the quality improvement effort and impact, undetected errors and rework cycles are unavoidable in complex development projects. With whatever rework is generated, it has its most destructive effect on the whole project when it is in the state of "undiscovered rework." Discovering the rework earlier and faster removes much of the program-wide disruption, especially the development time impacts.

To illustrate this, we used our model of the rework cycle to simulate the development time required for a project under varying levels of quality **and** the speed of rework discovery. Figure 2 summarizes the results in terms of the multiple of the original work schedule. (For convenience, think of a development effort planned to achieve initial work completion in one year—the results shown in Figure 2 would then be "number of years.")

The chart shows four lines—for rework discovery times equal to one-quarter of the original design plan time (one-fourth year in our example), as well as discovery times of one-half, three-quarters, and one.

As is now obvious, improvement either in "quality" or rework discovery yields much better schedule performance. It is noteworthy from the results charted that lowering the rework discovery time in an organization or project is most leveraged in improving schedule performance when quality is not at extremely low or extremely high levels. At extremely high quality levels, there simply isn't as much room for improvement of schedule performance. And in the stages of a development when extremely low quality prevails, rapid rework discovery ends up subjecting the execution of the discovered rework to the same low-quality conditions that caused it to cycle in the first place! *In such conditions, it is best to work first on quality enhancement practices and systems, then to accelerate the benefit with rework discovery enhancements*—such as earlier or improved reviews or testing.

MEASURING WHERE YOU STAND

The prevailing rework discovery time on design development efforts is typically in the range of one-fourth to three-fourths the scheduled length of the original design effort. In order to derive a good approximation of rework discovery times, construct a graph of the issues and reissues of the work product in the stage of development being examined (historically, if available; otherwise, monitor an ongoing effort). Graph over time each subsequent round of revision as a line. Simplified, your chart should look comparable to that in Figure 3.

By measuring the typical horizontal "distance" (time) between each adjacent pair of curves, good estimates of the rework discovery time (and how it changes over the project/stage) may be obtained.

Now you can also compute a good approximation of the time-varying "quality" of a project stage's work product.

Calculate the ratio of (a) the number of revisions to (b) the number of releases/revisions in the prior round; then subtract each computed ratio from 1.0. For example, if there were 350 "Revision B's" on 500 "Revision A's," the prevailing quality during the work on Revision A's was: 1 - (350/500) = 1 - 0.70 = 0.30.[4]

ARE YOU LOST IN THE TRIANGLE?

Armed with this new information, you will be able to assess where your project stands relative to other development projects. Further, you will be able to determine much more accurately where your project really stands **as it proceeds**.

We used our simulation model of the rework cycle to construct the charts in Figures 4 and 5, which are necessary for more correct mid-project assessments of progress and the magnitude of remaining effort. Using your estimates of project quality and rework discovery time to select the appropriate chart, one may "look up" the true (range of) real progress. In fact, with the benefit of data on a completed project, one may construct one's own "progress ramp" chart by plotting for the completed project: (1) the historically reported "percent complete," versus (2) a retrospective computation of the percent **really** complete then (you should compute the percent really complete based on hours spent to that point, relative to the total hours eventually spent).

Perfectly accurate project progress monitoring would yield a straight 45° diagonal (hence the triangular ramp shape): at a perceived/reported condition of 20 percent complete, the actual percent complete would be 20 percent, and so on. Instead, real progress is typically less than reported progress. Further, a given level of reported progress might mean any of a range of values for real progress as shown in Figure 4.

Shown in Figure 5 for each of three typical combinations of quality and rework discovery time are the relations between perceived percent complete, as reported by traditional systems, and the real percent complete, when undiscovered rework is taken account of. In every case there is a gap between real progress and that which is perceived. Looking across the three charts, it becomes clear that the **lower the quality and the longer the rework discovery times:**

- the larger the gap between real progress and that which is perceived, and the longer-lasting the gap;
- the later in the project/stage that a significant gap persists;
- the greater and longer-lasting the **uncertainty** in the size of the gap; and
- the later the point of maximum uncertainty about real progress.

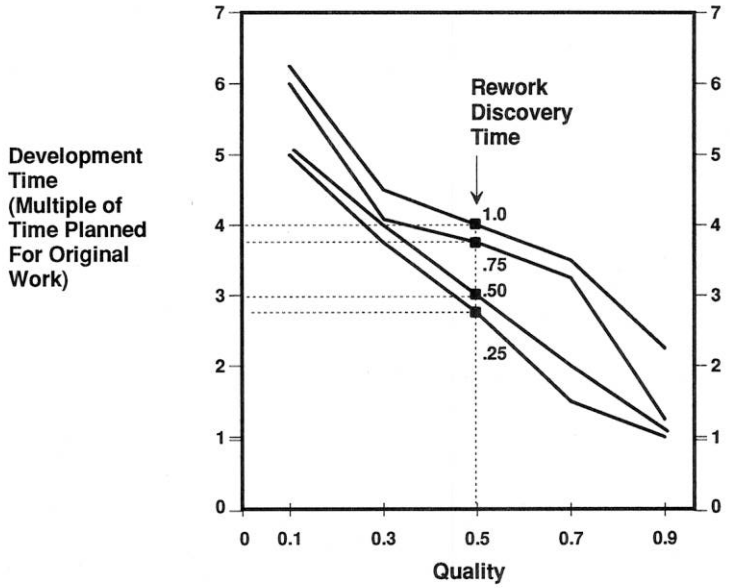


Figure 2. For most levels of work quality, improving the rework discovery time yields significant improvements in development schedules.

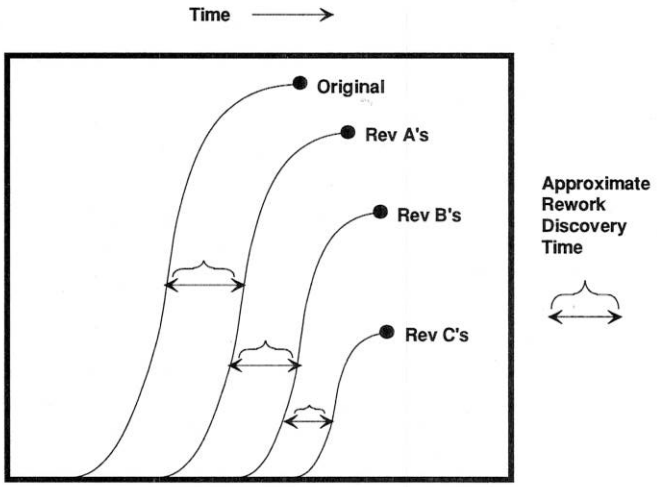


Figure 3. A critical addition to the set of closely monitored project performance measures should be the magnitude and timing of work revisions.

THE 90 PERCENT SYNDROME

The demonstrated uncertainty in real progress is responsible for several well-known troublesome project phenomena. One is the "90 Percent Syndrome," in which for a prolonged time project managers report to executives or the customer that their effort is "90 percent" done. Examine the middle progress ramp in Figure 5 for an example of this. In these conditions, an earnest, responsible manager might well report 90 percent progress achieved, when 75 or 80 percent is really done. And so it goes until, after much distress and disappointment (and time and cost), 90 percent

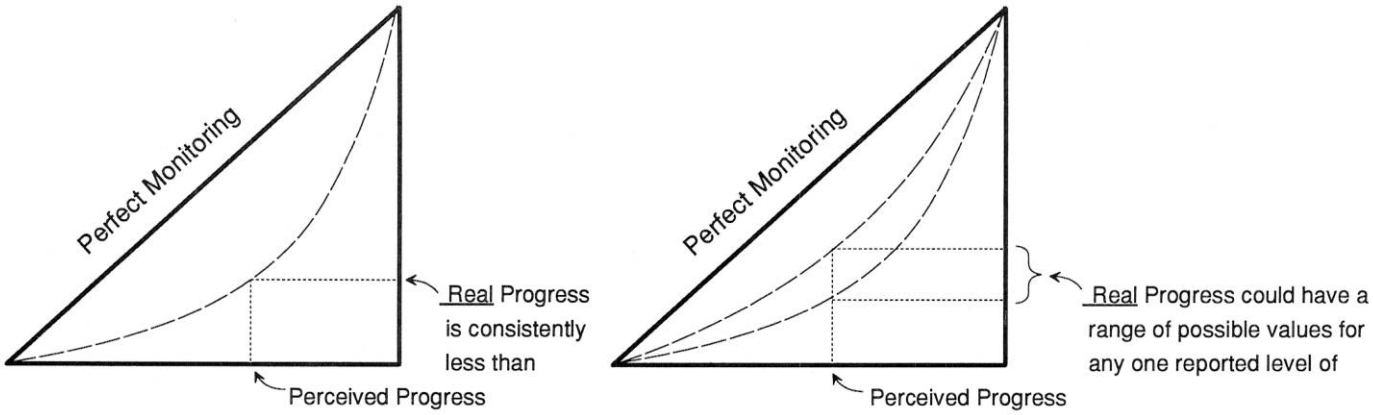


Figure 4. Progress Ramps help identify the true state of progress on a project or project phase.

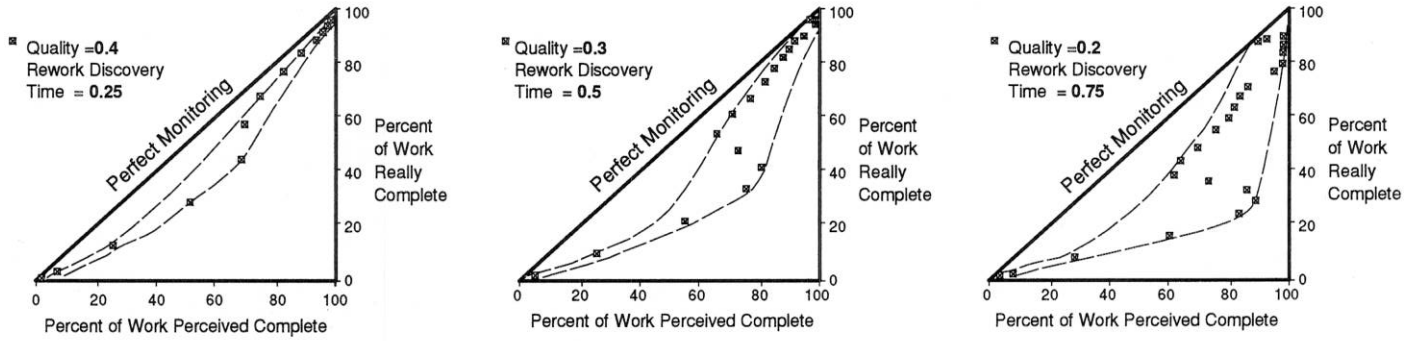


Figure 5. Lower "quality" and longer "rework discovery times" disguise low real progress, and increase the range of uncertainty in progress estimation.

is really achieved and the project moves on to completion. Quite apart from any lily-gilding inclinations of engineers, there is a systemic cause for the 90 percent syndrome.

THE LOST YEAR

One of our clients, managing a large new system development project described (not calmly) a related phenomenon, and requested a diagnosis. The 1000-person development effort had progressed to what seemed about two-thirds to three-quarters completion. **One year later**, after an additional thousand staff-years of effort, it seemed that they had made no progress, or had even lost ground, followed by a slow pace of progress toward completion. He called it the "Lost Year." His question: "What happened?" The abbreviated answer is clear once again from the middle progress ramp. In these conditions, 70 percent progress could be reported as "early" as at 30 percent real progress. Quite some time later (specifically, one year) having made 25-30 percent more real progress, "the system" can still be reporting 70 percent progress, or even less.

What happened? A lot of progress was made. A lot of rework was found and corrected (work that had been viewed as "done"). The "lost year" wasn't lost; it obviously felt like "one step forward, one step back," but it was just a large example of the rework cycle thwarting conventional monitoring systems. Even after the "lost" year, the project, still being reported at about 70 percent complete, was in fact just 60 percent complete, so what was seen as the remaining 30 percent felt like it took a long time to finish.

THE DELAYED PRODUCT INTRODUCTION

Not so very uncommon was the dilemma faced by another client firm. They wanted to introduce their new system product at the earliest responsible time in a technologically competitive market. But they had a history of undependable announcements on product release schedules, followed by dashed expectations. When could they really promise the market that this new system product would be available?

The diagnosis revealed a somewhat higher quality than in the prior charts, but with a longer rework discovery time, as characterized by the third progress ramp (shown again in Figure 6). In this condition the true status of a development project remains highly uncertain until the very end. Combined with consistent overestimation of progress are wide (vertical) bands of uncertainty within which perceived progress may become unexpectedly "stuck." While real progress is being made, the late and prolonged appearance of previously undiscovered rework results in ever-sliding, undependable estimates of completion time—particularly at the later stages, when companies are making announcements of new product introduction schedules.

This client's organization and practices had been set so as to encourage high levels of "completion" before subjecting to testing the integrated prototype. The analysis led the client to implement a new structure and set of procedures that included much more testing earlier. While this increased testing costs, total project time and costs were reduced significantly. And although the resulting earlier rework discovery was a bit "scary" to the managers

at first, much more dependable introduction schedules were attained.

CONCLUSION

Despite the need for advances in the stagnated theory and practice of development project management, no new gimmicks, no new software, will significantly improve development projects' performance. Rather, we need to improve our fundamental understanding of how projects really work. A healthy start is understanding the critical role of the *rework cycle*. It must be recognized, monitored and minimized—and if it is, we can eventually achieve dramatic breakthroughs in project costs and schedules.

Documented herein are the range of quality and rework in different projects, how to monitor quality and the rework discovery time, and how to translate these newly-collected measures into more accurate project progress assessments. They are necessary, but only the most basic steps toward a more strategic and realistic view of projects.

Even with "a better model," there will remain a managerial inertia forged from years, even decades, of misunderstanding. It will not be enough for project managers to internalize the associated lessons of the rework cycle. They must also learn the important differences between (a) the substantial *influence* the manager has over productivity, quality, and rework discovery (and hence, project costs and schedules) and (b) the relative lack of *control* the manager has over these same conditions. Understanding how the actions that managers can take influence their projects' outcomes requires extensions beyond the basic rework cycle structure. The very best of project managers know these intuitively. But just as important, managers' formulated action plans and their logic must be effectively and persuasively communicated to (and implemented with the aid of) many other players—senior company managers and colleagues, the project staff, partners and suppliers, and customers themselves. The passive manager (or worse, those who encourage obfuscating or ostrich-like behavior) won't make waves—nor will they make any contribution. Their projects will continue to fail. Successful project managers will take on the roles of thought leader and action leader—the advocate and instrument of change in the systems and practices that surround them. The alternative—the unmitigated machination of the rework cycle—will mean continuing the familiar pattern of development projects: unforeseen costs, unexpected delays, and unfulfilled promises.

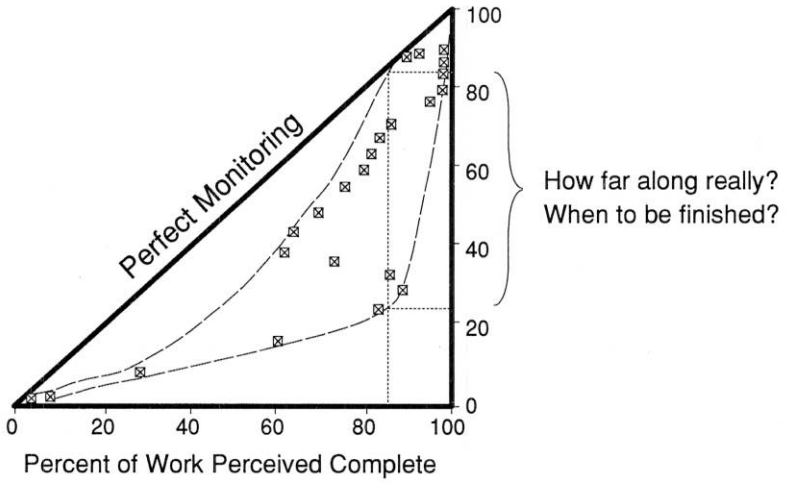


Figure 6. The end can appear to be near when real progress is as low as 30 percent, leading to overly-optimistic product introduction plans, destined to be delayed.

END NOTES

1. These project models were built using the dynamic continuous simulation language DYNAMO; see *DYNAMO User's Manual*. Pugh-Roberts Associates, and *Introduction to Systems Dynamics Modeling with DYNAMO*. Richardson, George P. and Pugh III, Alexander L.

2. We acknowledge the bias in the data caused by the fact that easy, smoothly-running projects rarely command the attention of consultants brought in by management. Therefore, the true range of "quality" values is likely to be broader "to the right" for each class if one includes less difficult projects.

3. Take heart, home builders: the construction projects in the sample reported here—power plants, combatant ships—all significantly exceed $1 billion.

4. Technical content and organizational practice in executing and reporting revisions vary widely. Account should be taken of the fact that the amount of effort on successive rounds of revisions will change (usually decrease). All measures of "quality" reported herein have been normalized to represent revision effort that is equal to original releases on a per unit basis.

Kenneth G. Cooper *is director of the Management Simulation Group and senior vice president of Pugh-Roberts Associates, a division of PA Consulting Group. His management consulting career spans 20 years, specializing in the development and application of computer simulation models to a variety of strategic business issues. His clients include AT&T, Arizona Public Service, Hughes Aircraft, IBM, Litton, McDonnell-Douglas, Northrop, Rockwell, Transmanche-Link, and several law firms. Mr. Cooper has directed over 100 consulting engagements, among them analyses of 60 major commercial and defense development projects. He is an original author of the program management model introduced in this article. His group's offices are in Cambridge, Massachusetts, and Oxford, England. Mr. Cooper received his bachelor's and master's degrees from M.I.T. and Boston University, respectively.*

This article is reprinted from the March 1993 *Project Management Journal* with permission of the Project Management Institute, Four Campus Boulevard, Newtown Square, PA 19073, USA, a worldwide organization for advancing the state of the art of project management. Phone 610/356-4600